



**NATIONAL RESEARCH UNIVERSITY OF INFORMATION
TECHNOLOGY, MECHANICS AND OPTICS**

FACULTY OF CONTROL SYSTEMS AND ROBOTICS

LABORATORY TASK N°4

COMPLETED BY: Valverde Ramírez, Edgar David

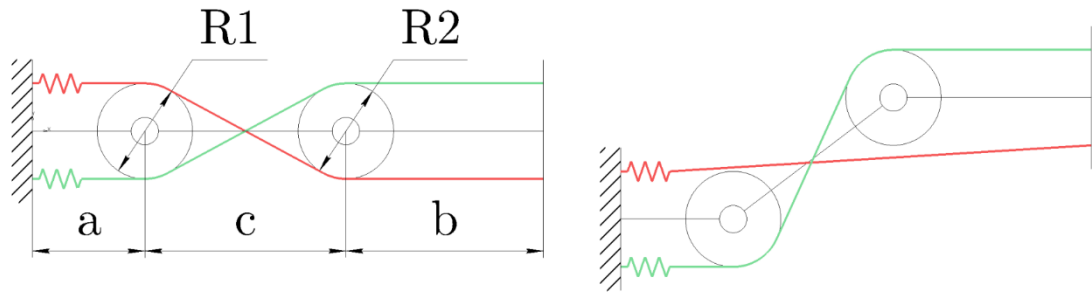
ISU: 520193

DISCIPLINE: Simulation of Robotic Systems

INSTRUCTOR: PhD. Ivan Borisov

1. Data:

The goal of this laboratory task is to implement and simulate an active control system for the previously modeled tendon-driven mechanism. Specifically, the task focuses on designing a Proportional-Derivative (PD) controller to ensure the joints follow a predefined sinusoidal trajectory within the MuJoCo physics engine.



The specific system parameters provided for the simulation are listed in the table below:

Parameter	Value
R1	0.01
R2	0.026
a	0.037
b	0.052
c	0.045

2. Modeling:

To simulate the mechanism, the system is defined in XML format. Based on the design schematics, the mechanism comprises a rigid base on the left side, followed by a chain of three links connected by two hinge joints. Cylinders are placed at the articulations to function as pulleys, and an end-effector is defined as the anchor point for the tendons.

Actuators

To enable control, `<motor>` tags were added to the XML configuration for each hinge joint (joint1 and joint2). These actuators allow the application of generalized forces (torques) directly to the degrees of freedom of the robot.

```
<actuator>
  <motor name="motor_q1" joint="joint1" gear="1"/>
  <motor name="motor_q2" joint="joint2" gear="1"/>
</actuator>
```

Sensors

To check if the motors manage to follow the expected behavior, speed and position sensors were added to each of the joints

```
<sensor>
  <jointpos name="sensor_q1" joint="joint1"/>
  <jointvel name="sensor_q1_vel" joint="joint1"/>
  <jointpos name="sensor_q2" joint="joint2"/>
  <jointvel name="sensor_q2_vel" joint="joint2"/>
</sensor>
```

3. Control

A Proportional-Derivative (PD) control law was implemented in the Python simulation loop to achieve trajectory tracking. The control signal τ for each joint is calculated as follows:

$$\tau_i = Kp_i(q_i^{des} - q_i) + Kd_i(\dot{q}_i^{des} - \dot{q}_i)$$

Where the equation for the desired position is:

$$q_i^{des} = AMP_i \sin(FREQ_i t) + BIAS_i$$

The target trajectory is a sinusoidal function for both joints, defined by specific amplitudes (AMP), frequencies (FREQ), and offsets (BIAS). The velocities are derived analytically as the derivative of the position function to ensure smooth tracking.

Parameter	Value
AMP_1	40.45°
AMP_2	26.52°
$FREQ_1$	2.93 Hz
$FREQ_2$	1.39 Hz
$BIAS_1$	-30.1°
$BIAS_2$	-27.5°

4. Python Simulation

The controller's performance was evaluated by comparing the desired trajectory (setpoint) against the real-time position of the joints.

```
with mujoco.viewer.launch_passive(model, data) as viewer:
    viewer.cam.azimuth = 90
    viewer.cam.elevation = -30
    viewer.cam.distance = 0.4
    viewer.cam.lookat = np.array([0.05, 0.0, 0.1])

    time.sleep(2)

    while viewer.is_running():
        step_start = time.time()

        t = data.time

        omega1 = 2 * np.pi * FREQ1
        q1_des = AMP1 * np.sin(omega1 * t) + BIAS1
        v1_des = AMP1 * omega1 * np.cos(omega1 * t)

        omega2 = 2 * np.pi * FREQ2
        q2_des = AMP2 * np.sin(omega2 * t) + BIAS2
        v2_des = AMP2 * omega2 * np.cos(omega2 * t)

        q1_act = data.qpos[model.joint('joint1').qposadr[0]]
        v1_act = data.qvel[model.joint('joint1').dofadr[0]]

        q2_act = data.qpos[model.joint('joint2').qposadr[0]]
        v2_act = data.qvel[model.joint('joint2').dofadr[0]]

        tau1 = Kp1 * (q1_des - q1_act) + Kd1 * (v1_des - v1_act)
        tau2 = Kp2 * (q2_des - q2_act) + Kd2 * (v2_des - v2_act)

        data.ctrl[0] = tau1
        data.ctrl[1] = tau2

        mujoco.mj_step(model, data)

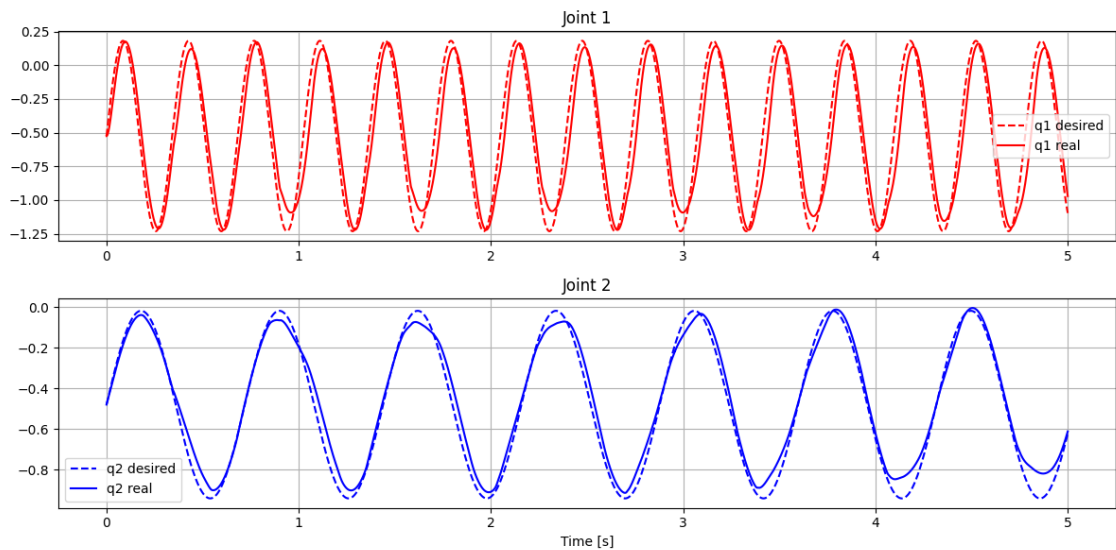
        history["t"].append(t)
        history["q1_des"].append(q1_des)
        history["q1_real"].append(q1_act)
        history["q2_des"].append(q2_des)
        history["q2_real"].append(q2_act)
```

```
viewer.sync()

time_until_next_step = model.opt.timestep - (time.time() -
step_start)
if time_until_next_step > 0:
    time.sleep(time_until_next_step)

if t > 5.0: break
```

5. Results:



6. Conclusions

Based on the implementation and results of the PD control simulation, the following conclusions are drawn:

- **Effectiveness of PD Control:** The Proportional-Derivative architecture proved sufficient for regulating the motion of the cable-driven mechanism. By providing both a corrective force and active damping, the controller successfully compensated for the system's inherent elasticity and inertia.
- **Trajectory Tracking Performance:** The system demonstrated the ability to track dynamic sinusoidal references. However, it was observed that as the reference frequency increases, the tracking error grows, indicating the physical limits of the actuators and the impact of the tendon-spring-damper dynamics on the system's bandwidth.

7. XML

```
<?xml version="1.0" ?>
<mujoco>
  <option timestep="1e-4" integrator="RK4" gravity="0 0 0"/>

  <default>
    <joint damping="0.001"/>
  </default>

  <worldbody>
    <light pos="0 0 3" dir="0 0 -1"/>
    <geom type="plane" size="1 1 0.1" rgba=".9 0 0 1"/>

    <body name="mid1" pos="0.063 0 0.1">
      <site name="mid1_site" pos="0 0 0" type="sphere" size="0.001"/>
      <joint name="mid1_x" type="slide" axis="1 0 0"/>
      <joint name="mid1_z" type="slide" axis="0 0 1"/>
      <geom type="sphere" size="0.002" rgba="1 0 0 1"/>
    </body>

    <body name="mid2" pos="0.063 0 0.1">
      <site name="mid2_site" pos="0 0 0" type="sphere" size="0.001"/>
      <joint name="mid2_x" type="slide" axis="1 0 0"/>
      <joint name="mid2_z" type="slide" axis="0 0 1"/>
      <geom type="sphere" size="0.002" rgba="0 1 0 1"/>
    </body>

    <body name="base" pos="0 0 0.1">
      <geom type="cylinder" size=".0005 .005" rgba=".8 .3 .3 1"/>
      <site name="anchor1a" pos="0 0 0.005" type="sphere"
size="0.002"/>
      <site name="anchor1b" pos="0 0 -0.005" type="sphere"
size="0.002"/>
      <body name="link1" pos="0 0 0">
        <geom name="link1_geom" type="cylinder" size=".0005 .0185"
fromto="0 0 0 0.037 0 0" rgba=".8 .3 .3 1" contype="0" conaffinity="0"/>
        <body name="link2" pos="0.037 0 0">
          <joint name="joint1" type="hinge" axis="0 1 0" pos="0 0
0"/>
          <geom name="link2_geom" type="cylinder" size=".0005
0.026" fromto="0 0 0 0.052 0 0" rgba=".3 .8 .3 1" contype="0"
conaffinity="0"/>

```

```

        <geom name="cyl1" type="cylinder" size=".005 .005"
euler="90 0 0" rgba=".3 .3 .3 0.5"/>
        <site name="wrap1a" pos="0 0 0.01" type="sphere"
size="0.001"/>
        <site name="wrap1b" pos="0 0 -0.01" type="sphere"
size="0.001"/>
        <site name="center1" pos="0 0 0" type="sphere"
size="0.001"/>
        <body name="link3" pos="0.052 0 0">
            <joint name="joint2" type="hinge" axis="0 1 0"
pos="0 0 0"/>
            <geom name="link3_geom" type="cylinder" size=".0005
.005" fromto="0 0 0 0.045 0 0" rgba=".3 .3 .8 1" contype="0"
conaffinity="0"/>
            <geom name="cyl2" type="cylinder" size=".013 .005"
euler="90 0 0" rgba=".3 .3 .3 0.5"/>
            <site name="wrap2a" pos="0 0 0.017" type="sphere"
size="0.001"/>
            <site name="wrap2b" pos="0 0 -0.017" type="sphere"
size="0.001"/>
            <site name="center2" pos="0 0 0" type="sphere"
size="0.001"/>
            <body name="end_effector" pos="0.045 0 0">
                <geom type="cylinder" size=".0005 .013" rgba=".3
.3 .8 1"/>
                <site name="anchor2a" pos="0 0 .013"
type="sphere" size="0.002"/>
                <site name="anchor2b" pos="0 0 -.013"
type="sphere" size="0.002"/>
                <site name="end_effector" pos="0 0 0"
type="sphere" size="0.001"/>
            </body>
        </body>
    </body>
</worldbody>

    <tendon>
        <spatial name="spring1" width="0.001" rgba="1 0 0 1" stiffness="50"
damping="10">
            <site site="anchor1a"/>
            <geom geom="cyl1" sidesite="wrap1a"/>
            <site site="mid1_site"/>
            <geom geom="cyl2" sidesite="wrap2b"/>

```

```

        <site site="anchor2b"/>
    </spatial>

    <spatial name="spring2" width="0.001" rgba="0 1 0 1" stiffness="50"
damping="10">
        <site site="anchor1b"/>
        <geom geom="cyl1" sidesite="wrap1b"/>
        <site site="mid2_site"/>
        <geom geom="cyl2" sidesite="wrap2a"/>
        <site site="anchor2a"/>
    </spatial>
</tendon>

<equality>
    <connect site1="mid1_site" site2="center1"/>
    <connect site1="mid1_site" site2="center2"/>
    <connect site1="mid2_site" site2="center1"/>
    <connect site1="mid2_site" site2="center2"/>
</equality>

<actuator>
    <motor name="motor_q1" joint="joint1" gear="1"/>
    <motor name="motor_q2" joint="joint2" gear="1"/>
</actuator>

<sensor>
    <jointpos name="sensor_q1" joint="joint1"/>
    <jointvel name="sensor_q1_vel" joint="joint1"/>
    <jointpos name="sensor_q2" joint="joint2"/>
    <jointvel name="sensor_q2_vel" joint="joint2"/>
</sensor>
</mujoco>

```

8. Python code

```

import mujoco
import mujoco.viewer
import matplotlib.pyplot as plt
import numpy as np
import time

def deg2rad(deg): return deg * (np.pi / 180.0)

AMP1 = deg2rad(40.45)

```

```

FREQ1 = 2.93
BIAS1 = deg2rad(-30.1)

AMP2 = deg2rad(26.52)
FREQ2 = 1.39
BIAS2 = deg2rad(-27.5)

Kp1 = 0.1
Kd1 = 0.001

Kp2 = 0.12
Kd2 = 0.01

model = mujoco.MjModel.from_xml_path("tendon.xml")
data = mujoco.MjData(model)

data.qpos[model.joint('joint1').qposadr[0]] = BIAS1
data.qpos[model.joint('joint2').qposadr[0]] = BIAS2

history = {"t": [], "q1_des": [], "q1_real": [], "q2_des": [], "q2_real":
[]}]

with mujoco.viewer.launch_passive(model, data) as viewer:
    viewer.cam.azimuth = 90
    viewer.cam.elevation = -30
    viewer.cam.distance = 0.4
    viewer.cam.lookat = np.array([0.05, 0.0, 0.1])

    time.sleep(2)

    while viewer.is_running():
        step_start = time.time()

        t = data.time

        omega1 = 2 * np.pi * FREQ1
        q1_des = AMP1 * np.sin(omega1 * t) + BIAS1
        v1_des = AMP1 * omega1 * np.cos(omega1 * t)

        omega2 = 2 * np.pi * FREQ2
        q2_des = AMP2 * np.sin(omega2 * t) + BIAS2
        v2_des = AMP2 * omega2 * np.cos(omega2 * t)

        q1_act = data.qpos[model.joint('joint1').qposadr[0]]
        v1_act = data.qvel[model.joint('joint1').dofadr[0]]

```

```

q2_act = data.qpos[model.joint('joint2').qposadr[0]]
v2_act = data.qvel[model.joint('joint2').dofadr[0]]

tau1 = Kp1 * (q1_des - q1_act) + Kd1 * (v1_des - v1_act)
tau2 = Kp2 * (q2_des - q2_act) + Kd2 * (v2_des - v2_act)

data.ctrl[0] = tau1
data.ctrl[1] = tau2

mujoco.mj_step(model, data)

history["t"].append(t)
history["q1_des"].append(q1_des)
history["q1_real"].append(q1_act)
history["q2_des"].append(q2_des)
history["q2_real"].append(q2_act)

viewer.sync()

time_until_next_step = model.opt.timestep - (time.time() -
step_start)
if time_until_next_step > 0:
    time.sleep(time_until_next_step)

if t > 5.0: break

plt.figure(figsize=(12, 6))

plt.subplot(2, 1, 1)
plt.plot(history["t"], history["q1_des"], 'r--', label="q1 desired")
plt.plot(history["t"], history["q1_real"], 'r', label="q1 real")
plt.title("Joint 1")
plt.legend()
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(history["t"], history["q2_des"], 'b--', label="q2 desired")
plt.plot(history["t"], history["q2_real"], 'b', label="q2 real")
plt.title("Joint 2")
plt.xlabel("Time [s]")
plt.legend()
plt.grid(True)

plt.tight_layout()

```

```
plt.show()
```